CPU Scheduling 2

Week 7

Operating Systems Ch 5 (Sections 5.3.4 - 5.8)

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

Part 1

Priority Scheduling (5.3.4)

Priority Scheduling

Each process is assigned a **priority (integer)**, and the process with the highest priority is executed first

- Generally, a **smaller number means higher priority** (per the textbook)
- Both **Preemptive** and **Nonpreemptive** are possible
  - Preemptive: preempts the current process when a higher priority process arrives
  - Nonpreemptive: waits until the current process voluntarily releases the CPU
- **SJF is a special case of Priority Scheduling**
  - Priority = inverse of the predicted next CPU burst

Priority Scheduling – Example

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

All processes arrive at time 0; smaller number means higher priority

```
| P2 | P5     | P1                | P3   | P4|
0    1        6                 16     18  19
```

Priority + Round-Robin

Among processes with the same priority, **RR** is applied

Process	Burst	Priority
P1	4	3
P2	5	2
P3	8	2
P4	7	1
P5	3	3

Time quantum = 2, smaller number means higher priority

Starvation and Aging

Starvation (Indefinite Blocking)

- A phenomenon where a low-priority process **never gets to execute** due to high-priority processes
- Anecdote: when MIT shut down an IBM 7094 in 1973, a low-priority job submitted in 1967 was discovered

Aging – Solution to Starvation

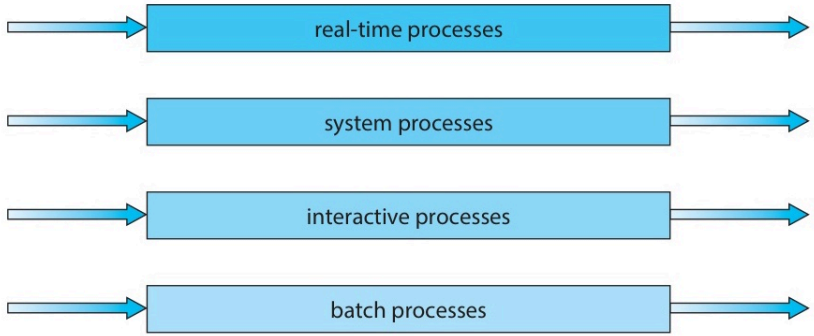
- **Gradually increase** the priority of waiting processes over time
- Example: in a priority range of 0(high)~127(low), increase priority by 1 every second
- A process with priority 127 reaches the highest priority in about **2 minutes**

Part 2

Multilevel Queue / MLFQ (5.3.5 - 5.3.6)

Multilevel Queue Scheduling

The ready queue is split into multiple **separate queues**



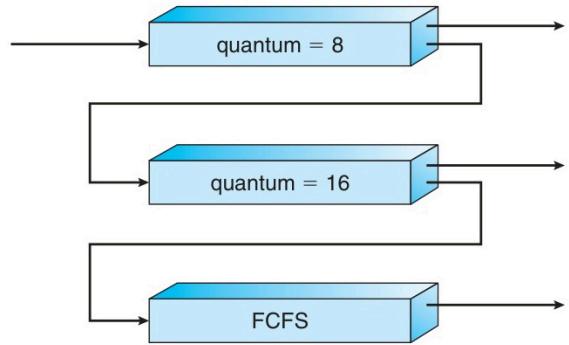
Silberschatz, Figure 5.8 – Multilevel queue scheduling

Scheduling between queues

- **Fixed priority**: lower queue executes only when upper queue is empty (starvation possible)
- **Time slice allocation**: CPU time assigned to each queue (e.g., foreground 80%, background 20%)
- Processes **cannot move** between queues (fixed assignment)

Multilevel Feedback Queue (MLFQ)

Difference from Multilevel Queue: **processes can move between queues**



Silberschatz, Figure 5.9 – Multilevel feedback queues

Movement rules

- **Demotion**: demoted to a lower queue when the allocated time quantum is fully consumed
- **Promotion (Aging)**: promoted to a higher queue when waiting too long in a lower queue

I/O-bound processes stay in upper queues due to **short bursts**, while CPU-bound processes move to lower queues

MLFQ Design Parameters

To define an MLFQ, the following must be determined

Parameter	Description
Number of queues	How many levels to use
Algorithm for each queue	RR, FCFS, etc.
Demotion criteria	When to demote to a lower queue
Promotion criteria	When to promote to a higher queue (Aging)
Initial queue assignment	Which queue to place a new process in

MLFQ Operation Example

Process P (CPU burst = 30ms) enters the system

```
Step 1: Runs 8ms in Q0 (RR, q=8)
        ⟶ remaining burst = 22ms, quantum exhausted ⟶ demoted to Q1

Step 2: Runs 16ms in Q1 (RR, q=16)
        ⟶ remaining burst = 6ms, quantum exhausted ⟶ demoted to Q2

Step 3: Runs 6ms in Q2 (FCFS)
        ⟶ completed
```

- Processes with burst <= 8ms: complete immediately in Q0 (highest priority)
- Processes with 8 < burst <= 24ms: go down only to Q1
- Processes with burst > 24ms: go down to Q2 (FCFS)

Favors short processes while ensuring long processes eventually execute

Part 3

Thread Scheduling (5.4)

Thread Scheduling – Contention Scope

Thread-level scheduling is divided into two **Contention Scopes**

PCS (Process-Contention Scope)

- **User-level thread** scheduling (handled by thread library)
- Competition among threads **within** a process
- Used in Many-to-One and Many-to-Many models

SCS (System-Contention Scope)

- **Kernel-level thread** scheduling (handled by OS kernel)
- Competition among **all** threads in the system
- Used in One-to-One model (**Linux, Windows**)

PCS vs SCS Comparison

Property	PCS	SCS
Scheduling entity	Thread library	OS kernel
Contention scope	Within process	System-wide
Thread model	Many-to-One, Many-to-Many	One-to-One
Priority adjustment	Set by programmer, not adjusted by library	Kernel can dynamically adjust
Time slicing	Not guaranteed (among same priority)	Guaranteed

Modern OSes (Linux, Windows, macOS) use **One-to-One model** -> **only SCS is supported**

Pthread Scheduling API

```c
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

int main(int argc, char *argv[]) {
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    pthread_attr_init(&attr);

    /* Query the current contention scope */
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
    else {
        if (scope == PTHREAD_SCOPE_PROCESS)
            printf("PTHREAD_SCOPE_PROCESS\n");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
            printf("PTHREAD_SCOPE_SYSTEM\n");
    }

    /* Set to SCS */
    pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
```

Part 4

Multi-Processor Scheduling (5.5)

Multiprocessor Architectures

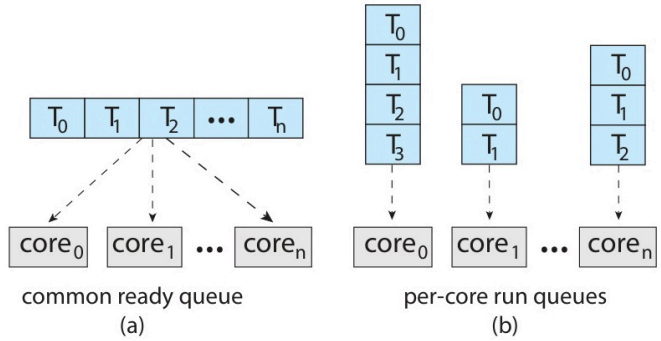
Scope of modern multiprocessors

- **Multicore CPUs** – multiple cores on a single chip
- **Multithreaded cores** – multiple hardware threads per core
- **NUMA systems** – Non-Uniform Memory Access
- **Heterogeneous multiprocessing (HMP)** – big.LITTLE, etc.

Two scheduling approaches:

- **Asymmetric Multiprocessing**: a single master processor makes all scheduling decisions (bottleneck)
- **Symmetric Multiprocessing (SMP)**: each processor schedules **independently**

SMP – Ready Queue Organization



Silberschatz, Figure 5.11 – Organization of ready queues

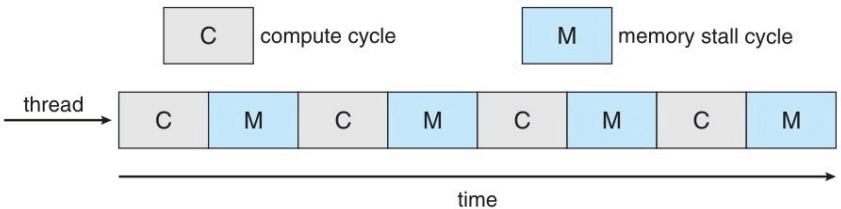
Approach	Pros	Cons
Common ready queue	Automatic load balancing	Lock contention, cache inefficiency
Per-processor queue	No locks, cache-friendly	Possible load imbalance

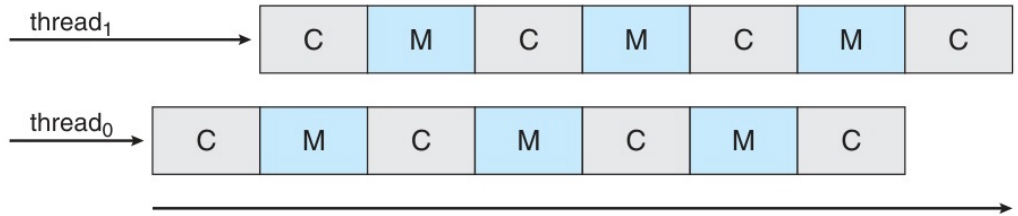
Modern SMP systems mostly use **per-processor queues**

Multicore Processors and Memory Stall

Memory Stall

- The time a processor waits for data to arrive when accessing memory
- Wastes tens to hundreds of cycles on a cache miss
- Can spend **up to 50% of time on memory wait**



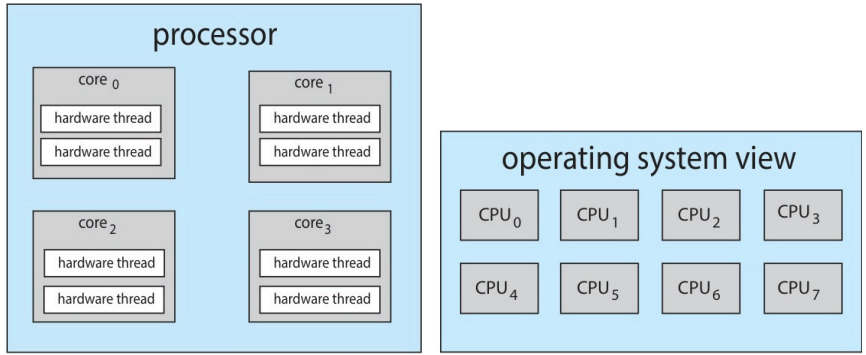


Silberschatz, Figure 5.12 – Memory stall & Figure 5.13 – Multithreaded multicore system

When one thread enters a memory stall, **another thread executes** -> increased core utilization

CMT (Chip Multithreading)

Multiple **hardware threads** placed on a single physical core



Silberschatz, Figure 5.14 – Chip multithreading

- Intel: **Hyper-Threading** (SMT) – 2 threads per core
- Oracle Sparc M7: 8 threads per core x 8 cores = **64 logical CPUs**

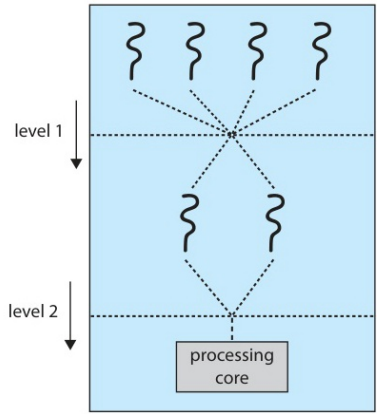
Each hardware thread appears as a **logical processor** to the OS

Coarse-grained vs Fine-grained Multithreading

Property	Coarse-grained	Fine-grained
Thread switch trigger	On long-latency events (memory stall, etc.)	At every instruction cycle boundary
Switch cost	**High** (pipeline flush required)	**Low** (switch logic built into HW)
Pros	Simple implementation	Fast switching, high core utilization
Cons	Pipeline flush overhead	Increased HW complexity

Two Levels of Scheduling

On multithreaded multicore processors, **two levels of scheduling** are needed



Silberschatz, Figure 5.15 – Two levels of scheduling

Level 1 (OS): assigns software threads to logical processors (HW threads) **Level 2 (Core)**: decides which hardware thread to switch to

- Round-robin approach (UltraSPARC T3)
- Dynamic urgency approach (Intel Itanium, 0~7 urgency value)

Processor Affinity

The tendency to **keep a process on a specific processor**

Why it is needed

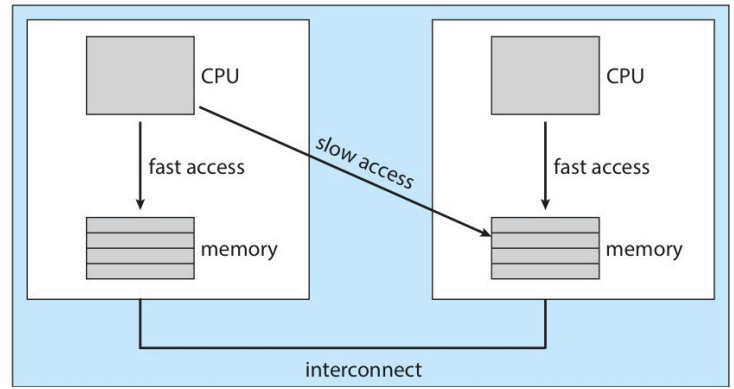
- Switching processors causes **cache invalidation**
- The cost of repopulating the cache on a new processor (**warm-up cost**)

Type	Description
Soft Affinity	OS attempts to keep the process on the same processor but **does not guarantee** it
Hard Affinity	Process **explicitly specifies** a particular processor set

Linux: supports **hard affinity** via the `sched_setaffinity()` system call

NUMA and Processor Affinity

NUMA (Non-Uniform Memory Access)



Silberschatz, Figure 5.16 – NUMA and CPU scheduling

- Each processor has nearby **local memory**
- Local memory access is **fast**, remote memory access is **slow**
- **NUMA-aware scheduling**: places processes on the processor where local memory resides
- CFS considers NUMA through a **scheduling domain** hierarchy

Load Balancing

When using per-processor queues, **load balancing** between processors is needed

Approach	Description
Push migration	Periodically checks each processor's load and moves tasks from overloaded to idle processors
Pull migration	An idle processor pulls tasks from a busy processor's queue

- It is common to **use both approaches together** (e.g., Linux CFS)
- **Load Balancing vs Processor Affinity**: a conflicting relationship
  - Balancing: wants to move processes to other processors
  - Affinity: wants to keep processes on the same processor

Heterogeneous Multiprocessing (HMP)

A combination of cores with the same instruction set but **different clock speeds and power consumption**

ARM big.LITTLE Architecture

- **big core**: high performance, high power -> suitable for short interactive tasks
- **LITTLE core**: low performance, low power -> suitable for background tasks

Scheduling strategy

- Interactive applications -> assigned to big cores
- Background tasks -> assigned to LITTLE cores
- Power-saving mode -> disable big cores, use only LITTLE cores

Unlike asymmetric multiprocessing: **all cores can run user/system tasks**

Part 5

Real-Time CPU Scheduling (5.6)

Soft vs Hard Real-Time

Category	Soft Real-Time	Hard Real-Time
Deadline	Misses allowed (performance degrades)	**Must be met** (system failure)
Guarantee	Guarantees priority-based execution	Guarantees completion within deadline
Examples	Multimedia playback, VoIP	Aircraft control, ABS, medical devices

Real-Time scheduling requirements

- Scheduler must be **preemptive**
- **Interrupt latency** and **dispatch latency** must be sufficiently short

Event Latency

Event Latency = time from event occurrence to service completion

```
event occurs                          system responds
     |←———— event latency ————→|
     t0                               t1
```

Two types of latency:

1. **Interrupt Latency**: interrupt arrival -> ISR start
   - Complete current instruction + determine interrupt type + context save
2. **Dispatch Latency**: time for the scheduler to perform process switching
   - **Conflict phase**: kernel preemption + resource release
   - **Dispatch phase**: assign high-priority process to CPU

Hard real-time: latency must be **bounded** (a few microseconds)

Periodic Task Model

Real-time processes are modeled as **periodic**

```
     p (period)           p                   p
|←————————————→|    |←————————————→|    |←————————————→|
|  d (deadline)|    |  d           |    |  d           |
|←————————→|   |    |←————————→|   |    |←————————→|   |
| t (burst)|   |    | t        |   |    | t        |   |
|←——→|   |   |    |←——→|   |   |    |←——→|   |   |
period 1        period 2        period 3
```

- **t**: processing time (CPU burst)
- **d**: deadline
- **p**: period
- Relationship: **0 <= t <= d <= p**
- Task rate = **1/p**
- CPU utilization = **t/p**

Rate-Monotonic Scheduling

Assigns **higher priority to tasks with shorter periods** (fixed priority, preemptive)

Rules

- Priority = 1 / period (shorter period means higher priority)
- Each task must complete execution within the deadline every period
- **Static priority**: priority does not change

Optimality

- **Optimal** among fixed-priority algorithms
- If a task set is not schedulable by Rate-Monotonic, it is **not schedulable by any fixed-priority algorithm**

Rate-Monotonic – Example (Success)

P1: period=50, burst=20 / P2: period=100, burst=35

- CPU utilization: 20/50 + 35/100 = 0.40 + 0.35 = **75%**
- Rate-Monotonic: P1 has higher priority (shorter period)

```
Deadlines: P1@50, P2@100, P1@100, P1@150, P2@200

 P1         P2                    P1    P2    idle  P1  ...
|####|──────|#####|#####|#####|──────|####|──────|#──|####|
0    20     25    35    45    50     70   75    80  100  120
     P1done  P2start               P1preempts  P2done
                                   P2resumes
```

```
|←── P1(20ms) ──→|←── P2(30ms) ──→| P1preempts |P2(5ms)|idle| P1  ...
0               20                 50       70     75    100
```

P1: runs 0-20, runs 50-70 --> all deadlines met P2: runs 20-50 (30ms), runs 70-75 (5ms) --> completes at 75ms, deadline 100 met

Rate-Monotonic – Example (Failure)

P1: period=50, burst=25 / P2: period=80, burst=35

- CPU utilization: 25/50 + 35/80 = 0.50 + 0.4375 = **93.75%**
- Theoretical upper bound for 2 tasks: N(2^(1/N)-1) = 2(2^0.5 - 1) = **82.8%**

```
|⟵ P1(25ms) ⟶|⟵ P2(25ms) ⟶| P1preempts |<P2(10ms)>|
0               25              50       75         85
                                                     ^
                                              P2 deadline = 80
                                              P2 misses deadline!
```

P1: runs 0-25 (done), runs 50-75 (done) P2: runs 25-50 (25ms), runs 75-85 (10ms) --> completes at 85ms P2's deadline is 80ms --> **deadline miss!**

CPU utilization 93.75% > upper bound 82.8% --> Rate-Monotonic cannot guarantee

Rate-Monotonic CPU Utilization Bound

Worst-case CPU utilization bound for N tasks:

U = N(2^(1/N) - 1)

N (number of tasks)	Utilization bound
1	100%
2	82.8%
3	78.0%
4	75.7%

Earliest-Deadline-First (EDF)

Executes the task with the **nearest deadline** first (dynamic priority, preemptive)

Key characteristics

- Priority **changes dynamically** based on deadlines
- Not limited to periodic tasks (aperiodic tasks are also possible)
- Theoretically schedulable up to **100%** CPU utilization
- However, due to context switching and interrupt handling costs, 100% is not achievable in practice

EDF – Example (Solving the Rate-Monotonic Failure Case)

P1: period=50, burst=25 / P2: period=80, burst=35

```
Time 0:  P1 deadline=50, P2 deadline=80  ⟶ P1 first
Time 25: P1 done, P2 starts
Time 50: P1 new period, P1 deadline=100, P2 deadline=80
          P2's deadline is closer ⟶ P2 continues!
Time 60: P2 done (35ms executed), P1 starts
Time 85: P1 done (25ms executed)
```

```
|⟵ P1(25ms) ⟶|⟵⟵⟵ P2(35ms) ⟶⟶⟶|⟵ P1(25ms) ⟶|
0              25                    60              85
```

- P1: deadline 50 met at time 25, deadline 100 met at time 85
- P2: deadline 80 met at time 60
- **All deadlines met!**

Rate-Monotonic vs EDF Comparison

Property	Rate-Monotonic	EDF
Priority	**Fixed** (period-based)	**Dynamic** (deadline-based)
CPU utilization bound	~69.3% (N->inf)	**100%** (theoretical)
Implementation complexity	**Low**	High
Overhead	**Low**	High (dynamic priority recalculation)
Task types	Periodic tasks only	Both periodic and aperiodic
Optimality	Optimal among fixed-priority	**Theoretically optimal**

Proportional Share Scheduling

Distribute a total of **T shares** among all applications

Example: T = 100 shares

- Process A: 50 shares -> **50%** of CPU time
- Process B: 15 shares -> **15%** of CPU time
- Process C: 20 shares -> **20%** of CPU time
- Remaining 15 shares -> surplus

Admission Control

- What if a new process requests 30 shares?
- In use: 50+15+20 = 85, remaining: 15 < 30 -> **rejected**

Admission only when sufficient shares are available -> provides **guaranteed proportional** CPU time

Part 6

Operating System Examples (5.7)

Linux CFS (Completely Fair Scheduler)

Default scheduler since Linux 2.6.23

History

- Before 2.5: traditional UNIX scheduler (poor SMP support)
- 2.5: **O(1) scheduler** (constant time, SMP support)
- 2.6.23~: **CFS** (improved interactive response)

Core goal: "completely fair" CPU time distribution

Scheduling Classes

- **Real-time class**: SCHED_FIFO, SCHED_RR (priority 0~99, higher priority)
- **Normal class**: **CFS** (priority 100~139, nice value -20~+19)

Linux Priority Ranges

Silberschatz, Figure 5.26 – Scheduling priorities on a Linux system

- **Smaller** number means higher priority
- Real-time tasks (0~99) always take **precedence** over normal tasks (100~139)
- Normal task priority = 100 + (nice + 20) = **120 + nice**

CFS – vruntime Concept

vruntime (virtual runtime): the virtual execution time of each task

- CFS runs the **task with the smallest vruntime** next
- Actual execution time is **weighted** based on nice value

nice value	vruntime increase rate
Low nice (high priority)	Increases **slower** than actual time
nice = 0 (default)	Increases at the **same rate** as actual time
High nice (low priority)	Increases **faster** than actual time

Example: task with nice=0 runs for 200ms -> vruntime = 200ms Example: task with nice=-5 runs for 200ms -> vruntime < 200ms

CFS – vruntime Operation Example

Task A (nice=0), Task B (nice=0), Task C (nice=5)

Initial vruntime: A=0, B=0, C=0

```
Step 1: A runs (10ms) → A.vruntime = 10
        B=0 < C=0 ≤ A=10 → B selected

Step 2: B runs (10ms) → B.vruntime = 10
        C=0 < A=10 = B=10 → C selected

Step 3: C runs (10ms) → C.vruntime = 15 (increases faster due to nice=5)
        A=10 = B=10 < C=15 → A selected

Step 4: A runs (10ms) → A.vruntime = 20
        B=10 < C=15 < A=20 → B selected
```

- Higher priority tasks have vruntime that increases **slowly** -> selected more frequently
- I/O-bound tasks maintain low vruntime due to short execution times -> naturally favored

CFS – Target Latency and Time Slice

CFS **does not use a fixed time quantum**

Target Latency

- Target time interval in which all runnable tasks execute **once**
- Example: target latency = 20ms, 4 tasks -> each task approximately 5ms

Minimum Granularity

- Prevents time slices from becoming extremely short when there are many tasks
- Example: minimum 1ms

Time Slice Calculation

- Target latency is distributed in **weighted proportions** based on nice values
- Higher priority tasks -> longer time slice
- Lower priority tasks -> shorter time slice

CFS – Red-Black Tree

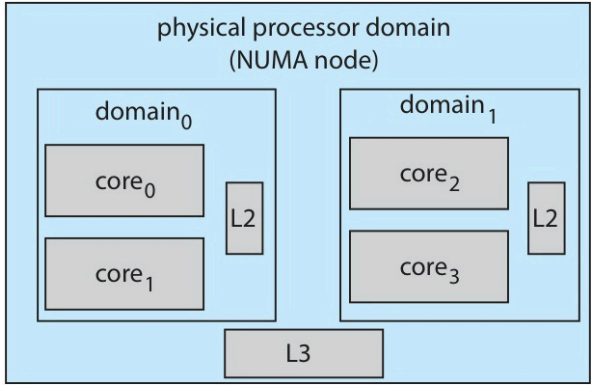
CFS stores runnable tasks in a **Red-Black Tree** (key = vruntime)

```
                  T3 (vruntime=25)
                 /                \
         T1 (vruntime=15)    T5 (vruntime=35)
         /        \              /        \
  T0 (vr=10)  T2 (vr=20)  T4 (vr=30)  T6 (vr=40)
   ^
   |
leftmost node
= smallest vruntime
= NEXT to run
```

- **Leftmost node** = smallest vruntime = next task to execute
- Cached in the **rb_leftmost** variable -> O(1) to determine the next task
- Insertion/deletion: **O(log N)**
- When a task blocks, it is **removed** from the tree; when it becomes runnable, it is **inserted** again

CFS – Load Balancing and NUMA

CFS load balancing utilizes a **scheduling domain hierarchy**



Silberschatz, Figure 5.27 – NUMA-aware load balancing with Linux CFS scheduler

Balancing strategy (from lower to higher levels)

1. Migration between cores within the same domain (shared L2, low cost)
2. Migration between domains (shared L3, medium cost)
3. Migration between NUMA nodes (high cost, **only under extreme imbalance**)

CFS defines **load = priority x CPU utilization ratio**

Windows Scheduling

Priority-based Preemptive Scheduling

Priority structure (32 levels)

- **0**: Idle thread (for memory management)
- **1~15**: Variable class
- **16~31**: Real-time class

Dispatcher: always selects the highest-priority ready thread

Behavior:

- Same priority -> **Round-Robin**
- Variable class: **priority decreases** when time quantum is exhausted
- **Priority boost** upon events such as I/O completion

Windows – Priority Classes

6 Priority Classes (Windows API)

Priority Class	Base Priority
REALTIME_PRIORITY_CLASS	24
HIGH_PRIORITY_CLASS	13
ABOVE_NORMAL_PRIORITY_CLASS	10
NORMAL_PRIORITY_CLASS	8
BELOW_NORMAL_PRIORITY_CLASS	6
IDLE_PRIORITY_CLASS	4

Windows – Variable Priority Behavior

```
Priority adjustment based on thread state changes

  Base Priority = 8 (NORMAL class, NORMAL relative)

  Time quantum exhausted
  ⟶ priority decreases (e.g., 8 → 7)
  ⟶ never drops below base priority

  I/O completion (keyboard)
  ⟶ priority boost (e.g., 8 → 14, large boost)

  I/O completion (disk)
  ⟶ priority boost (e.g., 8 → 10, medium boost)

  Foreground window
  ⟶ time quantum tripled
```

Goal: good response time for interactive threads, maintain I/O device utilization

Windows – Multiprocessor Support

SMT Set: groups hardware threads on the same core

- Example: dual-threaded/quad-core -> {0,1}, {2,3}, {4,5}, {6,7}
- Keeping threads within the same SMT set -> **cache efficiency**

Ideal Processor

- Each thread is assigned a **preferred processor** number
- Starts from the process's seed value, incremented on thread creation
- Distributed across different SMT sets: 0, 2, 4, 6, 0, 2, …
- Different seeds per process -> even distribution overall

Part 7

Algorithm Evaluation (5.8)

Scheduling Algorithm Evaluation Methods

Method	Description	Accuracy
Deterministic Modeling	Analytical calculation with a specific workload	Low (specific cases only)
Queueing Models	Mathematical models (Little's formula, etc.)	Medium (approximation)
Simulations	Computer simulation	High
Implementation	Implement in actual OS and measure	Highest (real environment)

Higher accuracy means **higher cost and complexity**

Deterministic Modeling

Assume a specific workload and calculate the exact performance of each algorithm

Process	Burst Time
P1	10
P2	29
P3	3
P4	7
P5	12

All processes arrive at time 0

Queueing Models – Little's Formula

Model the system as a network of **servers + queues**

Little's Formula:

n = lambda x W

- **n**: average queue length (number of waiting processes)
- **lambda**: average arrival rate (processes/second)
- **W**: average waiting time (seconds)

Example: lambda = 7 processes/sec, n = 14 processes

- W = n / lambda = 14/7 = **2 seconds**

Limitations: distributions that are mathematically intractable, limitations of independence assumptions

Simulations and Implementation

Simulation

- Program a software model of the computer system
- Data sources: random number generation or **trace files** (actual system records)
- Trace files: reproduce actual event sequences exactly -> high accuracy
- Cons: development cost, execution time, storage space

Implementation

- Implement the algorithm in an actual OS and run it
- The **only completely accurate** evaluation method
- Cons: implementation/testing cost, user behavior changes with environment changes
- Users may change their behavior to match the algorithm (gaming)

Lab

Multilevel Feedback Queue Simulator

Lab – MLFQ Simulator

Implement a 3-level Multilevel Feedback Queue simulator

Queue Configuration

Queue	Algorithm	Time Quantum
Q0	Round-Robin	8
Q1	Round-Robin	16
Q2	FCFS	None (until completion)

Rules

- New processes enter Q0
- If not completed within Q0's quantum -> demoted to Q1
- If not completed within Q1's quantum -> demoted to Q2

Lab – Implementation Guide

```python
from collections import deque

class Process:
    def __init__(self, pid, arrival, burst):
        self.pid = pid
        self.arrival = arrival
        self.burst = burst
        self.remaining = burst
        self.queue_level = 0
        self.wait_time = 0
        self.start_time = -1
        self.finish_time = -1

def mlfq_scheduler(processes, quanta=[8, 16]):
    queues = [deque(), deque(), deque()]
    time = 0
    completed = []
    gantt = []

    while not all_done(processes, completed):
        # 1. Insert arriving processes into Q0
        # 2. Check aging: promote processes waiting too long in Q2 to Q0
        # 3. Select process in order Q0 → Q1 → Q2
        # 4. Execute according to the selected queue's algorithm
```

Lab – Core Logic Implementation

```python
def run_process(proc, queue_level, quanta, time):
    """Run the selected process for the queue's quantum"""
    if queue_level < 2:  # Q0 or Q1
        quantum = quanta[queue_level]
        run_time = min(proc.remaining, quantum)
    else:  # Q2 (FCFS)
        run_time = proc.remaining

    proc.remaining -= run_time
    new_time = time + run_time

    if proc.remaining == 0:
        # Process completed
        proc.finish_time = new_time
        return new_time, 'completed'
    else:
        # Quantum exhausted → demotion
        proc.queue_level = min(queue_level + 1, 2)
        return new_time, 'demoted'
```

Test data: P1(0,30), P2(2,12), P3(4,6), P4(6,20), P5(8,4)

Lab – Sample Output

Gantt Chart

```
| P1(Q0) | P2(Q0) | P3(Q0) | P4(Q0) | P5(Q0) | P2(Q1) | P4(Q1) | P1(Q1) | P1(Q2) |
0        8       16       22       30       34       38       50       66       72
```

Statistics

Process	Arrival	Burst	Finish	Turnaround	Waiting
P1	0	30	72	72	42
P2	2	12	38	36	24
P3	4	6	22	18	12
P4	6	20	50	44	24

Summary

Key concepts this week

Topic	Key Points
Priority Scheduling	Priority-based, Starvation solved with **Aging**
Multilevel Queue	Split into multiple queues, different algorithms per queue, **fixed assignment**
MLFQ	**Movement between queues** possible, most general-purpose scheduling
Thread Scheduling	PCS (intra-process competition) vs SCS (system-wide competition)
SMP / Affinity	Per-processor queue, **Soft/Hard affinity**, NUMA-aware
Load Balancing	**Push/Pull migration**, conflicts with affinity

Next Week

Week 8: Midterm Exam

Scope

- Ch 1~2: Introduction, OS Structures
- Ch 3: Processes
- Ch 4: Threads and Concurrency
- Ch 5: CPU Scheduling

Exam Types

- Conceptual essay questions
- Algorithm calculation problems (Scheduling, Gantt chart, etc.)
- Code analysis (fork, thread, IPC, etc.)
- Compare/contrast questions (Rate-Monotonic vs EDF, PCS vs SCS, etc.)